

NFE204: BASE DE DONNEES AVANCEES (1)

Projet

Recherche d'information, classification, regroupement avec SOLR

Application à une collection d'articles scientifiques

KHENNICHE SADIA

2015

1 TABLE DES MATIERES

2	Introduction	1
3	Objectif.....	1
4	Collections.....	1
4.1	Processus de collecte des données:.....	1
4.1.1	Export depuis HAL.....	2
4.1.2	Export depuis IEEE	2
4.1.3	Import vers MongoDB	2
4.2	Configuration du moteur de recherche	2
4.2.1	Export depuis MongoDB	2
4.2.2	Configuration de Solr	3
4.2.3	Insertion des données dans l'index Solr	4
5	Résultats.....	5
5.1	Recherche plein texte	5
5.2	Regroupement	6
5.2.1	Affichage des résultats.....	6
5.2.2	Paramètres de regroupement	7
5.2.3	Comparaison des algorithmes	7
5.2.4	Application de filtre	9
5.2.5	Utilisation des mots clés	9
6	Discussion.....	10
7	Conclusion.....	10
8	Annexes.....	11
8.1	Annexe 1 : extraction des documents depuis HAL.....	11
8.2	Annexe 2 : extraction des documents depuis IEEE	13
8.3	Annexe 3 : résultat d'une recherche plein texte.....	15
8.4	Annexe 4 : interface de carrot2 workbench	16

2 INTRODUCTION

L'essor de la publication ou du partage d'information scientifique sur internet conduit à des possibilités de recherche d'information bibliographique importantes.

Dans le cadre du projet pour l'unité d'enseignement NFE204, il est proposé d'utiliser ces dépôts d'articles et de communications scientifiques pour effectuer une recherche sur l'état de l'art pour un domaine scientifique.

Une base de données NoSQL ainsi qu'un moteur de recherche sont utilisés dans le cadre de ce projet.

3 OBJECTIF

A partir d'une collection d'articles scientifiques, il s'agit d'avoir un panorama de l'état de l'art sur la recherche dans le domaine des bases de données informatiques, tout en étudiant les possibilités de recherche, classement et regroupement que propose le moteur de recherche SOLR.

La recherche bibliographique est focalisée sur les notions abordées dans les unités d'enseignement NFE204 et NFE205 du CNAM, au cours de la spécialisation « bases de données » du diplôme d'ingénieur CNAM en informatique-systèmes d'information (CYC12).

4 COLLECTIONS

L'étude porte sur des collections de documents extraits de deux sites bibliographiques : HAL (<https://hal.archives-ouvertes.fr>) et IEEE (<http://ieeexplore.ieee.org>).

4.1 PROCESSUS DE COLLECTE DES DONNEES:

La base de données est constituée une première fois, les données pouvant être mises à jour à une fréquence mensuelle, par import différentiel des nouvelles données.

Afin de constituer les collections, un export est effectué depuis les sites internet des dépôts d'articles, puis les données sont importées sur une base MongoDB.

HAL fournit plusieurs formats d'export, dont le format « json » et « csv », tous deux importables dans MongoDB avec la commande « mongoimport ». IEE propose le format csv, mais pas le format json.

Il a été choisi d'effectuer l'export en csv, pour utiliser la même méthode.

La volumétrie utile est de quelques Mégaoctets. L'ensemble des deux collections comporte environs 5000 enregistrements.

4.1.1 Export depuis HAL

Le site HAL permet de consulter les dépôts par discipline. Pour notre sujet, la discipline « informatique » avec la sous-catégorie « bases de données » sont sélectionnées. Un dernier filtre est effectué pour n'obtenir que les dépôts de type « Article dans des revues » et « Communication dans un congrès ». Au vu de la limite à 2000 enregistrements par export, les deux listes d'articles sont exportées l'une après l'autre.

L'annexe 1 illustre le processus d'extraction des données depuis HAL.

Les champs choisis comportent au moins l'auteur, l'année de publication, la revue, le titre, le résumé, les mots clés et l'URI de la ressource.

4.1.2 Export depuis IEEE

Pour explorer les articles déposés sur IEEE pour la discipline des bases de données, il a été choisi de se focaliser sur la revue « Knowledge and Data Engineering, IEEE Transactions on » (publication numéro 29). Au vu de la limite à 2000 enregistrements par export, des filtres successifs sont appliqués sur la date de publication. L'annexe 2 illustre le processus d'extraction des données depuis IEEE.

Les champs utiles peuvent être sélectionnés dans les fichiers csv exportés.

4.1.3 Import vers MongoDB

Après avoir créé les collections « hal » et « iee » dans la base MongoDB « nfe204 », les commandes suivantes ont été utilisées pour importer les données:

```
mongoimport -d nfe204 -c hal --type csv --file HAL_discipline_database_article.csv --headerline
mongoimport -d nfe204 -c hal --type csv --file HAL_discipline_database_congres.csv --headerline

mongoimport -d nfe204 -c iee --type csv --file Knowledge_and_Data_Engieniering_1989_1999.csv --headerline
mongoimport -d nfe204 -c iee --type csv --file Knowledge_and_Data_Engieniering_2000_2009.csv --headerline
mongoimport -d nfe204 -c iee --type csv --file Knowledge_and_Data_Engieniering_2010_2015.csv --headerline
```

4.2 CONFIGURATION DU MOTEUR DE RECHERCHE

4.2.1 Export depuis MongoDB

Les commandes suivantes ont été utilisées pour exporter les données:

```
mongoexport -d nfe204 -c hal -f "uri_s,title_s,keyword_s,en_abstract_s" -o hal_for_solr.json
mongoexport -d nfe204 -c iee -f "PDF Link,Document Title,Author Keywords,Abstract" -o iee_for_solr.json
```

Ensuite, en vue de la fusion des deux collections dans le même index Solr, le fichier export de « iee » est modifié : les clés sont renommées avec les noms "uri_s, title_s, keyword_s, en_abstract_s".

Remarque : le fichier de format json pour la collection HAL aurait pu être généré directement depuis le site internet de HAL. Il a été produit depuis MongoDB par souci d'homogénéité entre les collections, et sur le principe que MongoDB est utilisé pour stocker la base documentaire, qui

pourra a posteriori recevoir des documents provenant de sites bibliographiques supplémentaires.

4.2.2 Configuration de Solr

Il s'agit de mettre en place les index (« core » dans le langage Solr) qui permettront la recherche d'information sur les documents.

Un index commun à toutes les sources bibliographique est créé avec le nom « bibli ».

L'index est configuré en fonction des champs décrivant les documents. Le fichier schema.xml permet la définition des champs et des analyseurs. Lors de la définition des champs, les champs contenant le résumé, les mots clés, et le titre des articles sont copiés dans un champ consolidé, appelé « text », pour la recherche plein texte.

Le fichier solrconfig.xml permet, entre autres, de paramétrer le mécanisme de regroupement (clustering) des documents.

Des extraits de ces deux fichiers sont montrés ci-dessous.

Extrait de la configuration schema.xml, relatif à la définition des champs :

```
<schema name="example" version="1.5">
<!-- Liste des champs de l'index -->
  <fields>
    <field name="_id" type="string" indexed="true" stored="true"
      required="false" multiValued="true"/>
    <field name="abstract_s" type="text_general" indexed="true" stored="true"
      required="false" />
    <field name="keyword_s" type="text_general" indexed="true" stored="true"
      required="false" />
    <field name="title_s" type="text_general" indexed="true" stored="true"
      required="false" />
    <field name="uri_s" type="text_general" indexed="true" stored="true"
      required="false" />

    <!-- Un champ dans lequel on concatène les autres pour une recherche "plein-texte" -->
    <field name="text" type="text_general" indexed="true" stored="false"
      multiValued="true" />
    <copyField source="abstract_s" dest="text" />
    <copyField source="title_s" dest="text" />
    <copyField source="keyword_s" dest="text" />

    <!-- Un champ "technique" requis par Solr/Lucene -->
    <field name="_version_" type="long" indexed="true" stored="true" />
  </fields>

  <!-- La clé d'accès à un document dans l'index -->
  <uniqueKey>uri_s</uniqueKey>
```

Extrait de la configuration schema.xml, relatif à l'analyseur des champs textes :

```
<fieldType name="text_general" class="solr.TextField" positionIncrementGap="100">
  <analyzer type="index">
    <tokenizer class="solr.StandardTokenizerFactory"/>
```

```

<filter class="solr.StopFilterFactory" ignoreCase="true" words="stopwords_fr.txt" />
<!-- in this example, we will only use synonyms at query time
<filter class="solr.SynonymFilterFactory" synonyms="index_synonyms.txt" ignoreCase="true" expand="false"/>
-->
<filter class="solr.LowerCaseFilterFactory"/>
    <!-- added by Sadia K. -->
    <filter class="solr.ASCIIFoldingFilterFactory"/>
    <filter class="solr.SnowballPorterFilterFactory" language="English"/>
</analyzer>
<analyzer type="query">
    <tokenizer class="solr.StandardTokenizerFactory"/>
    <filter class="solr.StopFilterFactory" ignoreCase="true" words="stopwords.txt" />
    <!-- Sadia : trouver un fichier de synonymes français
    <filter class="solr.SynonymFilterFactory" synonyms="index_synonyms.txt" ignoreCase="true" expand="false"/>
    -->
    <filter class="solr.LowerCaseFilterFactory"/>
    <!-- added by Sadia K. -->
    <filter class="solr.ASCIIFoldingFilterFactory"/>
    <filter class="solr.SnowballPorterFilterFactory" language="English"/>
</analyzer>
</fieldType>

```

Extrait de la configuration solrconfig.xml, relatif au choix des champs pour le regroupement:

```

<!-- Field name with the logical "title" of a each document (optional) -->
<str name="carrot.title">title_s</str>
<!-- Field name with the logical "URL" of a each document (optional) -->
<str name="carrot.url">uri_s</str>
<!-- Field name with the logical "content" of a each document (optional) -->
<str name="carrot.snippet">text</str>
<!-- Apply highlighter to the title/ content and use this for clustering. -->
<bool name="carrot.produceSummary">true</bool>

```

4.2.3 Insertion des données dans l'index Solr

Les commandes suivantes ont été utilisées pour insérer les données dans l'index « bibli »:

```

curl http://localhost:8983/solr/bibli/update/json?commit=true --data-binary @hal_for_solr.json -H "Content-type:application/json"
curl http://localhost:8983/solr/bibli/update/json?commit=true --data-binary @ieee_for_solr.json -H "Content-type:application/json"

```

5 RESULTATS

5.1 RECHERCHE PLEIN TEXTE

Une liste de concepts étudiés en cours NFE204 et NFE205 est établie :

agglomeration, aggregation, analyzer, bags of features, big data, bucket, classification, cloud, clustering, cosine similarity, dimension curse, distribution, full text, hashing, index, invariance, inverse document frequency, json, k means, knn, lemmatization, map-reduce, metric space, most informative, most positive, nosql, partition, precision recall, replication, scalability, semantic, similarity, stemming, stop word, term frequency, tf-idf, token, tree, xml, web

La recherche est effectuée avec l'interface graphique de Solr, avec le mode debugQuery pour comprendre le calcul des scores.

Un exemple de résultat est donné en annexe 3. Une recherche a été effectuée sur l'expression « bag of features », concept utilisé en recherche multimédia.

On observe bien que l'analyseur a supprimé le mot vide « of », et pris la racine de « features ». Le calcul de score se fait alors sur la chaîne "bag ? featur".

Seul deux documents sont trouvés sur les 5000 de la bibliographie. Les scores sont calculés avec le produit de trois facteurs constituant le poids du terme :

- la fréquence du terme (tf) dans le document (en fait Solr prend la racine carrée de la fréquence) : plus le terme est présent, plus élevé est le score,
- la fréquence inverse du terme dans la collection (idf) : plus est discriminant le terme, plus sa présence dans un document est significative, faisant augmenter le score,
- la norme (field norm) du document (Solr prend l'inverse de la racine carrée du nombre de termes) : moins il y a de termes dans le document, plus est significative la présence du terme recherché, faisant augmenter le score.

Dans le cas d'une recherche avec plusieurs termes (par exemple si l'on recherche bag of features sans mettre les guillemets autour de l'expression entière, les mots « bag » et « features » sont cherchés indépendamment, et leur score ajouté), deux autres facteurs interviennent :

- le poids de la requête, constituée de l'idf et de la norme de la requête,
- coord : proportion de termes recherchés présents dans le document.

Le tableau ci-dessous compare les résultats obtenus pour les recherches d'information sur « bag of features » et « big data » :

requête	nombre de documents trouvés	score du premier document	score du dernier document	idf de la phrase	tf dans le premier document	norme inverse du premier document
"bag of features"	2	1.49	0.90	9.56	2.00	0.08
"big data"	37	4.48	0.39	7.18	1.00	0.62

La documentation sur le concept de big data est plus la plus abondante (37 documents). Il est à noter que le score très élevé du premier document s'explique par le fait qu'il ne comporte ni mot clé, ni résumé, et que le seul texte qui le compose est un titre aussi laconique que possible sur le sujet : il s'intitule « big data ».

5.2 REGROUPEMENT

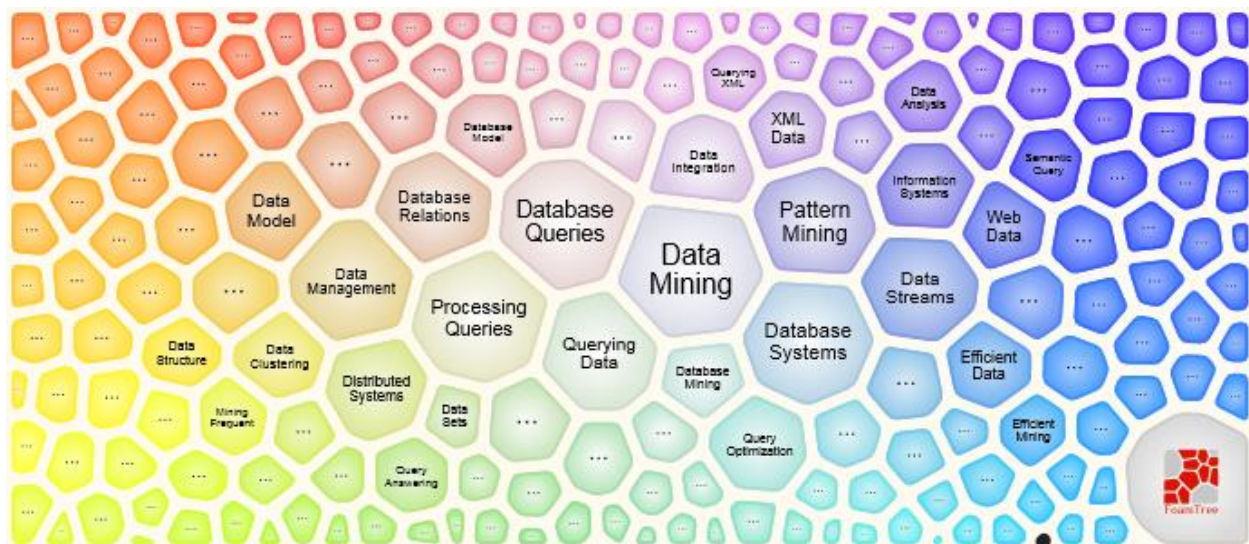
Solr fournit une intégration du moteur de regroupement Carrot2. Les tests de regroupements ont été effectués depuis la console d'administration Solr, puis depuis un client Carrot2, «Carrot2 workbench », qui a l'intérêt de disposer les résultats sous forme graphique.

5.2.1 Affichage des résultats

Voici un extrait de résultat présenté en JSON sur l'interface Solr admin :

```
"clusters": [
  {
    "labels": [
      "Data Mining"
    ],
    "score": 4.8729070439393185,
    "docs": [
      (liste de 103 documents)
    ]
  },
  {
    "labels": [
      "Processing Queries"
    ],
    "score": 10.69958126024404,
    "docs": [
      (liste de 90 documents)
    ]
  },
  {
    "labels": [
      "Database Queries"
    ],
    "score": 3.6458093069633137,
    "docs": [
      (liste de 86 documents)
    ]
  },
  ...
]
```


Voici l'affichage graphique du même regroupement sur Carrot2 workbench :



L'annexe 4 montre une vision globale l'interface de Carrot2 workbench.

5.2.2 Paramètres de regroupement

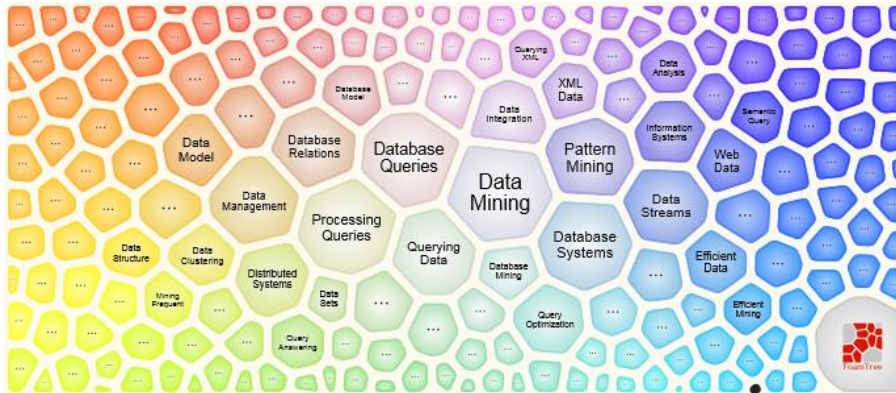
De nombreux paramètres sont disponibles, qui font varier significativement les résultats. Certains sont listés ci-dessous :

- algorithme de regroupement: les trois principaux sont Lingo, k-mean et STC ; Lingo 3G est aussi proposé sous licence payante,
- nombre de documents,
- filtre,
- champs de contenu, de titre et d'url,
- paramètres de l'algorithme Lingo : nombre de groupes, seuil de fusion de clusters chevauchants, etc.,
- paramètres de l'algorithme k-mean : nombre de groupes, nombres de labels par cluster, nombre d'itération, etc.,
- paramètres de l'algorithme STC : nombre de groupes, nombre de phrases par label, etc.

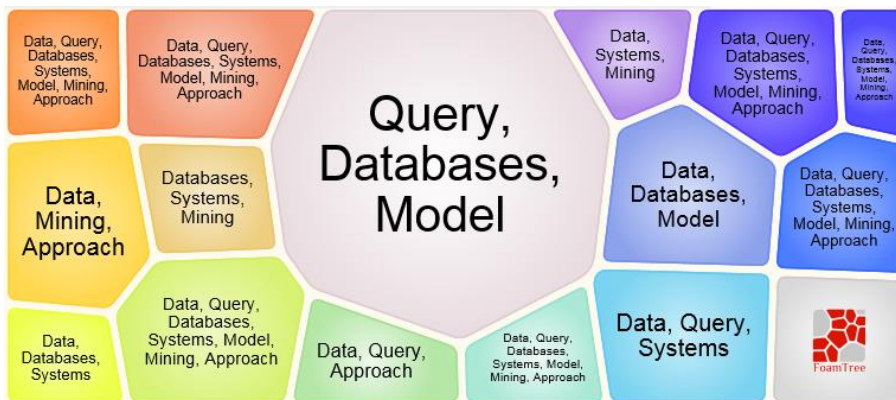
5.2.3 Comparaison des algorithmes

Les copies d'écran ci-dessous montrent les résultats obtenus pour chacun des algorithmes, avec 5000 documents et le champ de contenu défini par le champ Solr « text » (concaténation du titre, du résumé et des mots clés des articles). Tous les paramètres des algorithmes sont laissés par défaut pour cette comparaison.

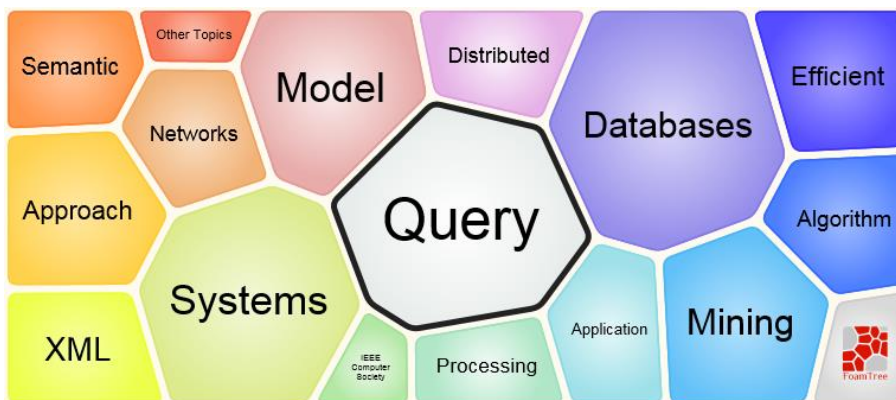
Algorithme Lingo :



Algorithme k-mean :



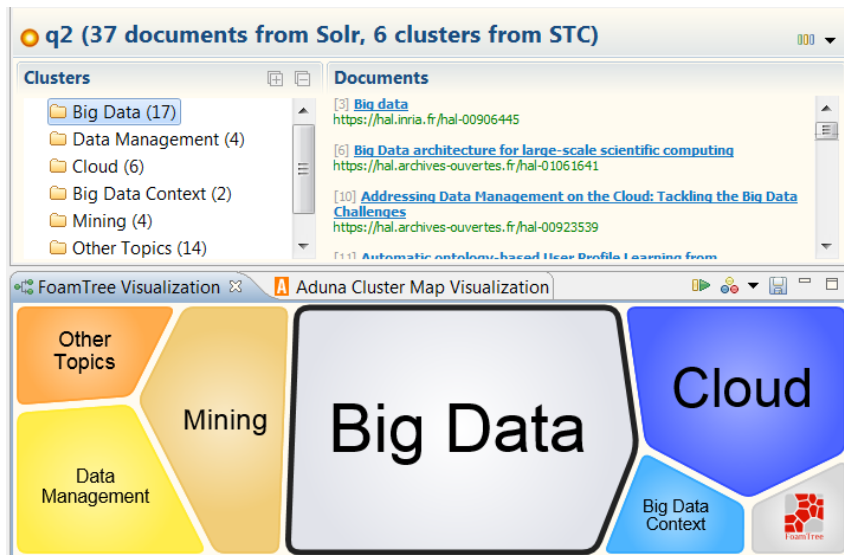
Algorithme STC :



L'algorithme STC semble le plus pertinent pour notre collection de document.

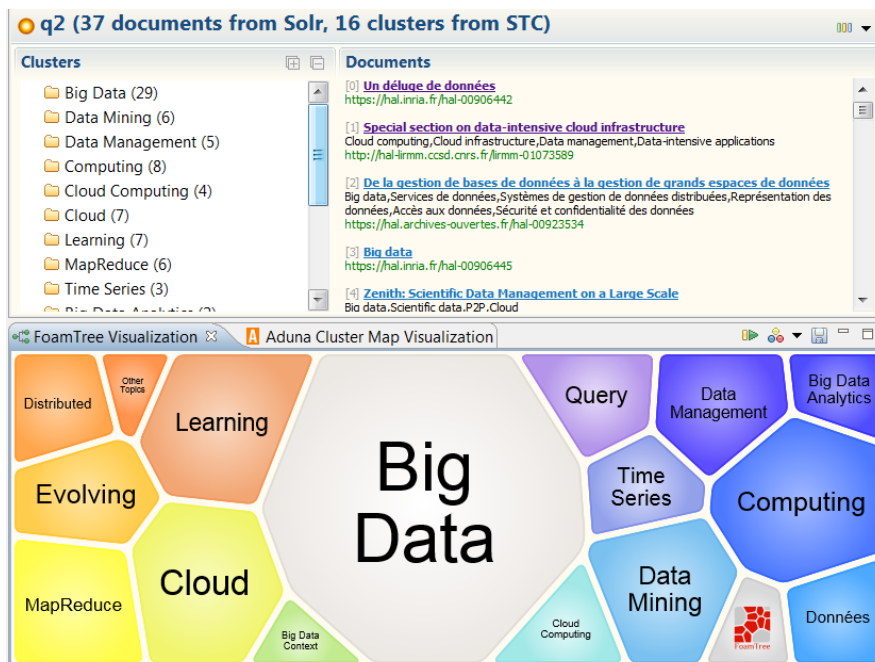
5.2.4 Application de filtre

Le regroupement STC est testé en appliquant un filtre avec l'expression « big data », utilisée précédemment lors des recherches plein texte. Nous obtenons les 37 documents attendus, répartis en 6 groupes, comme le montre la copie d'écran ci-dessous.



5.2.5 Utilisation des mots clés

Le champ « test », qui est la somme des champs de titre, de mots clés, et de résumé, a été utilisé jusqu'à présent comme champ prioritaire pour effectuer les regroupements. Nous testons à présent le champ des mots clés (« keyword_s »). Le regroupement apparaît plus fin :

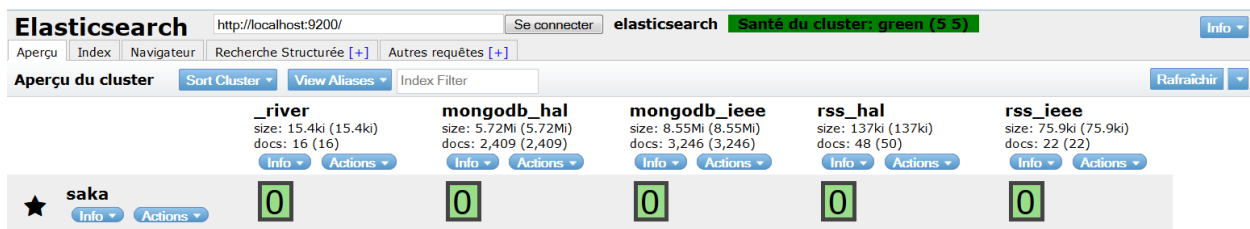


6 DISCUSSION

Solr est un moteur de recherche puissant permettant d'effectuer sans problème des recherches plein texte avec des fonctionnalités avancées comme le regroupement.

Bien que la concaténation de tous les champs textes dans un seul puisse permettre une recherche plein texte efficace sur l'ensemble du document, la désignation du seul champ contenant les mots clé produit des résultats plus intéressants pour le regroupement. Dans une collection de documents spécialisés, la présence des mots clés pour décrire les documents reste donc très utile.

Comparé à Elasticsearch, Solr pêche par l'absence de processus de synchronisation de ses index avec la source de données. Elasticsearch propose des rivières MongoDB et RSS qui ont pu être testées pour les bases bibliographiques HAL et IEE, comme l'illustre la copie d'écran ci-dessous.



Néanmoins, il semble que dans la configuration matérielle utilisée (ordinateur personnel), les rivières RSS rendaient Elasticsearch très instable.

Un sujet d'approfondissement pourrait être de comparer le regroupement de Solr (fonctionnalité appelée clustering) avec celui d'Elasticsearch (fonctionnalité appelée aggregation).

7 CONCLUSION

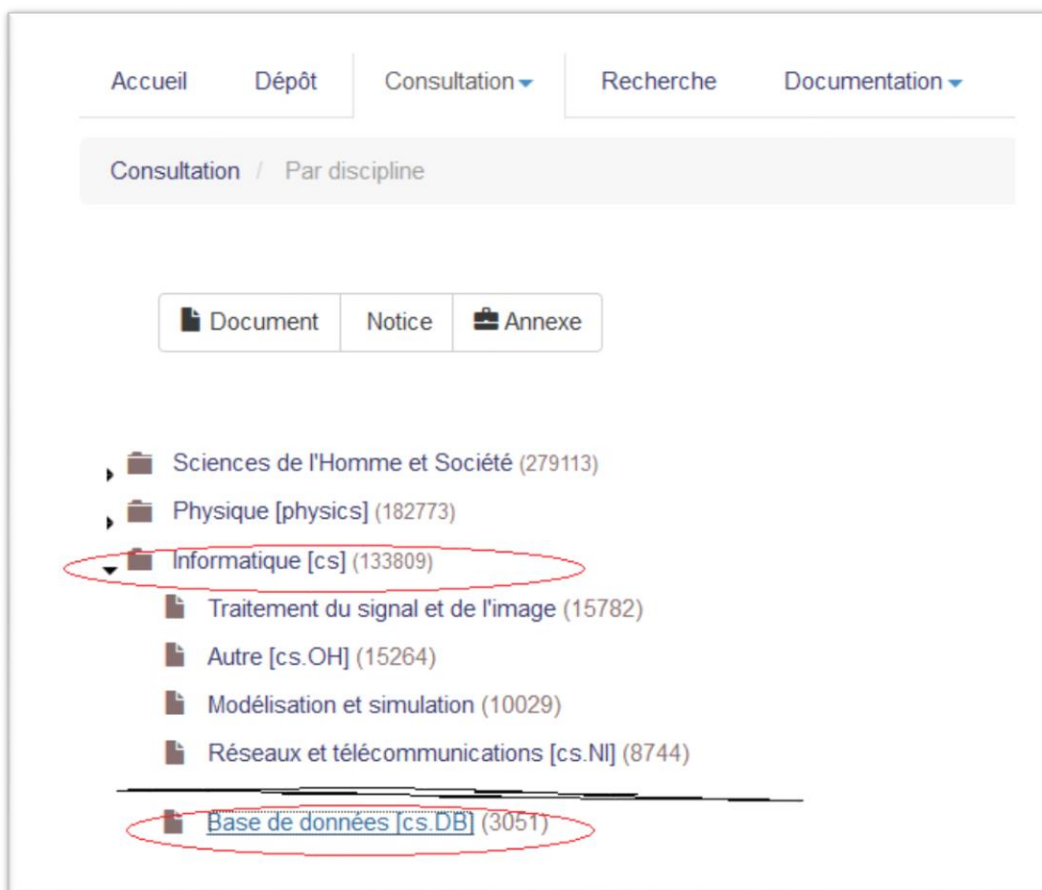
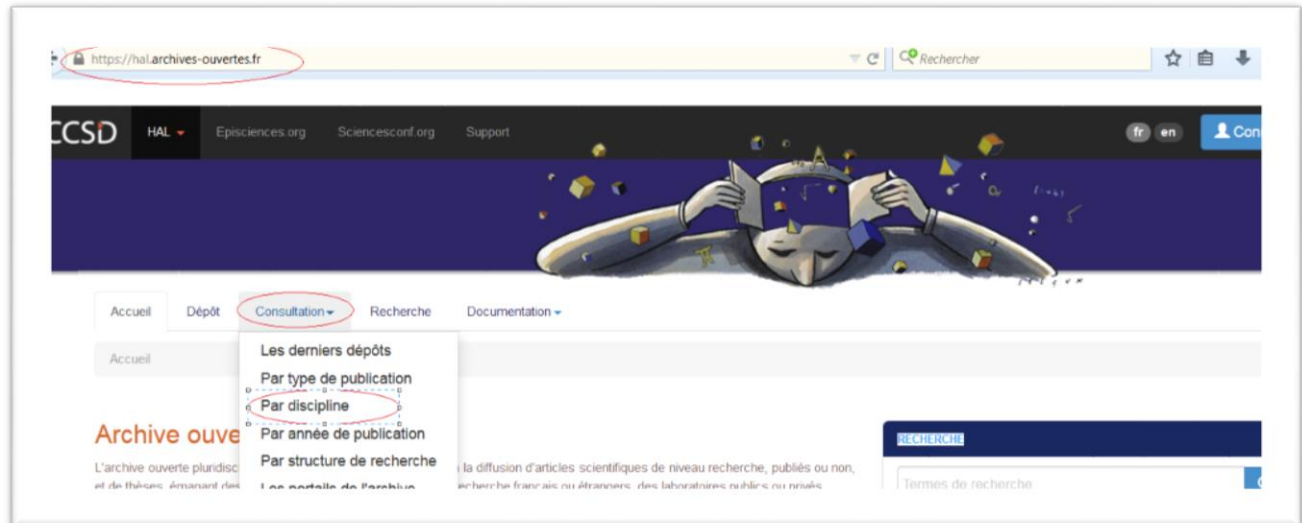
Lors de ce projet, une ébauche de base documentaire bibliographique a été produite à partir de deux sources documentaires, avec un focus sur la discipline des bases de données informatiques.

Les collections mises en place peuvent être complétées par d'autres ressources, afin de permettre de dresser un état de l'art dans le domaine des bases de données.

Ce projet a ainsi constitué pour moi une façon de prolonger ce cycle de cours en bases de données avancées, par des perspectives de veille technologique.

8 ANNEXES

8.1 ANNEXE 1 : EXTRACTION DES DOCUMENTS DEPUIS HAL



é

8.2 ANNEXE 2 : EXTRACTION DES DOCUMENTS DEPUIS IEEE

IEEE.org | IEEE Xplore Digital Library | IEEE-SA | IEEE Spectrum | More Sites

IEEE Xplore®
Digital Library

> Institutional Sign In

BROWSE ▼ MY SETTINGS ▼ GET HELP ▼ WHAT CAN I ACCESS? SU

Advanced Search Options

Advanced Keyword/Phrases Command Search Citation Search Preferences

ENTER KEYWORDS OR PHRASES, SELECT FIELDS, AND SELECT OPERATORS
Note: Refresh page to reflect updated preferences.

Search : ☒ Metadata Only ☐ Full Text & Metadata ?

69 in Publication Number

AND in Metadata Only

Select All on Page Download Citations ▼ Export to IEEE Collaborative

Refine results by ?

Search within results

Content Type

☐ Journals & Magazines (3,586)
☐ Early Access Articles (92)

Year

Single Year Range

1900 2015

☐ **[Front cover]**
Knowledge and Data Engineering, IEEE Transactions on
Year: 2004, Volume: 16, Issue: 10
Pages: c1 - c1, DOI: 10.1109/TKDE.2004.54
IEEE Journals & Magazines
PDF (147 Kb) ©

☐ **Attribute-level neighbor hierarchy construction using evolved pattern-based knowledge induction**
Puthongsiriporn, T.; Porter, J.D.; Bidanda, B.; Wang, M.-E.; Billo, R.E
Knowledge and Data Engineering, IEEE Transactions on
Year: 2006, Volume: 18, Issue: 7
Pages: 917 - 929, DOI: 10.1109/TKDE.2006.104
IEEE Journals & Magazines
Abstract (html) PDF (3960 Kb) ©

Displaying results 1-25 of 2,943 for ("Publication Number":69) x and refined by
Year: 2000-2015 x

Show All Results | Per Page 25 | Sort By Relevance

☐ Select All on Page [Download Citations](#) ☐ Export to IEEE Collabratec

Refine results by

Search within results

Content Type

- ☐ Journals & Magazines (2,851)
- ☐ Early Access Articles (92)

☐ [Front cover] Knowledge ar
Year: 2004, V
Pages: c1 - c
IEEE Journal
PDF (147 Kb)

☐ Attribute-level pattern-base
Puthongsirip
Knowledge and Data Engineering, IEEE Transactions on

If no search results are selected, the top 2000 results will be downloaded (CSV only).

Output Format

- ☐ Plain Text
- ☐ BibTeX
- ☐ RefWorks
- ☐ RIS (EndNote, Reference Manager, ProCite)
- ☒ CSV

Include

- ☐ Citation Only
- ☒ Citation & Abstract

Cancel Download

8.3 ANNEXE 3 : RESULTAT D'UNE RECHERCHE PLEIN TEXTE



Dashboard

Logging

Core Admin

Java Properties

Thread Dump

bibli

Overview

Analysis

Dataimport

Documents

Files

Ping

Raw Query Parameters

key1=val1&key2=val2

wt

xml

☒ indent

☒ debugQuery

☐ dismax

☐ edismax

☐ hl

☐ facet

☐ spatial

☐ spellcheck

Execute Query

Request-Handler (qt)

/select

common

q

"bag of features"

fq

sort

start, rows

0 10

fl

df

Raw Query Parameters

key1=val1&key2=val2

http://localhost:8983/solr/bibli/select?q=%22bag+of+features%22%0A&wt=xml&indent=true&debugQuery=true

<?xml version="1.0" encoding="UTF-8"?>
<response>

<lst name="responseHeader">
 <int name="status">0</int>
 <int name="QTime">3</int>
 <lst name="params">
 <str name="q">"bag of features"
 </str>
 <str name="indent">true</str>
 <str name="wt">xml</str>
 <str name="debugQuery">true</str>
 <str name="_">1441026129860</str>
 </lst>
</lst>
<result name="response" numFound="2" start="0">
 <doc>
 <str name="abstract_s">One of the main limitations of image search based on bag-of-features is the memor
 <str name="keyword_s">Data compression,Image coding,Indexing</str>
 <str name="title_s">Packing bag-of-features</str>
 <str name="uri_s">https://hal.inria.fr/inria-00394213</str>
 <long name="_version_">1511020214588801025</long></doc>

 <str name="abstract_s">Burstiness\, a phenomenon initially observed in text retrieval\, is the property
 <str name="keyword_s">Reference datasets,Visual elements,Burstiness,Text retrieval,Visual bursts,Image
 <str name="title_s">On the burstiness of visual elements</str>
 <str name="uri_s">https://hal.inria.fr/inria-00394211v3</str>
 <long name="_version_">1511020214597189633</long></doc>
 </result>
 <lst name="debug">
 <str name="rawquerystring">"bag of features"
 </str>
 <str name="querystring">"bag of features"
 </str>
 <str name="parsedquery">PhraseQuery(text:"bag ? featur")</str>
 <str name="parsedquery_toString">text:"bag ? featur"</str>
 <lst name="explain">
 <str name="https://hal.inria.fr/inria-00394213">
1.4936875 = (MATCH) weight(text:"bag ? featur" in 2404) [DefaultSimilarity], result of:
 1.4936875 = fieldWeight in 2404, product of:
 2.0 = tf(freq=4.0), with freq of:
 4.0 = phraseFreq=4.0
 9.5596 = idf(), sum of:
 6.242276 = idf(docFreq=29, maxDocs=5673)
 3.317324 = idf(docFreq=558, maxDocs=5673)
 0.078125 = fieldNorm(doc=2404)

 3.317324 = idf(docFreq=558, maxDocs=5673)
 0.078125 = fieldNorm(doc=2404)
 </str>
 <str name="https://hal.inria.fr/inria-00394211v3">
0.89621246 = (MATCH) weight(text:"bag ? featur" in 2417) [DefaultSimilarity], result of:
 0.89621246 = fieldWeight in 2417, product of:
 1.0 = tf(freq=1.0), with freq of:
 1.0 = phraseFreq=1.0
 9.5596 = idf(), sum of:
 6.242276 = idf(docFreq=29, maxDocs=5673)
 3.317324 = idf(docFreq=558, maxDocs=5673)
 0.09375 = fieldNorm(doc=2417)
 </str>
 </lst>
</lst>

8.4 ANNEXE 4 : INTERFACE DE CARROT2 WORKBENCH

The screenshot displays the Carrot2 Workbench interface, which is used for clustering and analyzing search results. The interface is divided into several panels:

- Search Panel (Top Left):** Shows the source as "Solr" and the algorithm as "Lingo". It includes a "Query (required)" field with the value "h2", a checkbox for "Read Solr clusters if present", and a "Results" field with the value "5000". There is also a checkbox for "Use highlighter output if present" and a "Medium" dropdown.
- Documents Panel (Top Right):** Displays a list of search results, each with a title and a URL. The results are numbered from [15] to [773].
- Visualization Panel (Bottom Left):** Shows a hexagonal grid visualization of the search results, with each hexagon representing a document. The grid is color-coded, with red and orange hexagons at the top and green and blue hexagons at the bottom.
- Attributes Panel (Bottom Right):** Contains configuration settings for the clustering process. It includes:
 - Clusters:**
 - Cluster count base: 20
 - Cluster merging threshold: 0.70
 - Size-Score sorting ratio: 0.00
 - Labels:**
 - Cluster label assignment method (required): Unique Label Assigner
 - Phrase label boost: 1.50
 - Phrase length penalty start: 8
 - Phrase length penalty stop: 8
 - Remove labels ending in genitive form: ☒
 - Remove leading and trailing stop words: ☒
 - Remove numeric labels: ☒